

1 第 2 题:ordinary renewal process 的更新函数统计量

设普通更新过程的更新间隔为

$$X_1, X_2, \dots,$$

并假设它们独立同分布, 分布函数为 F . 记

$$S_k = X_1 + \dots + X_k, \quad S_0 = 0,$$

则时刻 t 前的更新次数为

$$N(t) = \max\{k : S_k \leq t\}.$$

这里 $N(t)$ 就是 renewal counting function. 因为

$$\{N(t) = k\} = \{S_k \leq t < S_{k+1}\},$$

所以它的分布可以写成

$$\mathbb{P}(N(t) = k) = F^{*k}(t) - F^{*(k+1)}(t), \quad k = 0, 1, 2, \dots, \quad (1)$$

其中 F^{*k} 表示 F 的 k 重卷积, 并约定 $F^{*0}(t) = 1$.

因此, $N(t)$ 的 n 阶原点矩为

$$\mathbb{E}[N(t)^n] = \sum_{k=0}^{\infty} k^n (F^{*k}(t) - F^{*(k+1)}(t)). \quad (2)$$

这个公式是最直接的精确表达式.

也可以用下降阶乘矩写得更紧凑. 记

$$(x)_r = x(x-1) \cdots (x-r+1),$$

则

$$\mathbb{E}[(N(t))_r] = r! \sum_{k=r}^{\infty} \binom{k-1}{r-1} F^{*k}(t), \quad r = 1, 2, \dots \quad (3)$$

这是因为当 $N(t) = m$ 时, 右端内部的组合数满足

$$\sum_{k=r}^m \binom{k-1}{r-1} = \binom{m}{r}.$$

再利用第二类 Stirling 数 $\left\{ \begin{smallmatrix} n \\ r \end{smallmatrix} \right\}$, 有

$$N(t)^n = \sum_{r=1}^n \left\{ \begin{smallmatrix} n \\ r \end{smallmatrix} \right\} (N(t))_r. \quad (4)$$

于是得到

$$\mathbb{E}[N(t)^n] = \sum_{r=1}^n \left\{ \begin{smallmatrix} n \\ r \end{smallmatrix} \right\} r! \sum_{k=r}^{\infty} \binom{k-1}{r-1} F^{*k}(t). \quad (5)$$

特别地, 更新函数的期望为

$$m(t) := \mathbb{E}[N(t)] = \sum_{k=1}^{\infty} F^{*k}(t). \quad (6)$$

二阶矩为

$$\mathbb{E}[N(t)^2] = \mathbb{E}[N(t)] + \mathbb{E}[N(t)(N(t) - 1)] = \sum_{k=1}^{\infty} F^{*k}(t) + 2 \sum_{k=2}^{\infty} (k-1)F^{*k}(t). \quad (7)$$

因此方差为

$$\text{Var}(N(t)) = \sum_{k=1}^{\infty} F^{*k}(t) + 2 \sum_{k=2}^{\infty} (k-1)F^{*k}(t) - \left(\sum_{k=1}^{\infty} F^{*k}(t) \right)^2. \quad (8)$$

2 第 3 题: 兴奋-抑制神经网络模型的 Hopf 分叉

考虑系统

$$\tau_E \frac{dv_E}{dt} = -v_E + [M_{EE}v_E - M_{EI}v_I + h_E]_+, \quad (9)$$

$$\tau_I \frac{dv_I}{dt} = -v_I + [M_{IE}v_E - M_{II}v_I + h_I]_+, \quad (10)$$

其中 $[z]_+ = \max\{z, 0\}$. 先在两个输入都为正的区域分析, 即

$$M_{EE}v_E - M_{EI}v_I + h_E > 0, \quad M_{IE}v_E - M_{II}v_I + h_I > 0.$$

此时系统化为线性系统

$$\tau_E \dot{v}_E = (M_{EE} - 1)v_E - M_{EI}v_I + h_E, \quad (11)$$

$$\tau_I \dot{v}_I = M_{IE}v_E - (1 + M_{II})v_I + h_I. \quad (12)$$

其平衡点由

$$\begin{pmatrix} M_{EE} - 1 & -M_{EI} \\ M_{IE} & -(1 + M_{II}) \end{pmatrix} \begin{pmatrix} v_E^* \\ v_I^* \end{pmatrix} + \begin{pmatrix} h_E \\ h_I \end{pmatrix} = 0 \quad (13)$$

给出 Jacobian 矩阵为

$$J = \begin{pmatrix} \frac{M_{EE} - 1}{\tau_E} & -\frac{M_{EI}}{\tau_E} \\ \frac{M_{IE}}{\tau_I} & -\frac{1 + M_{II}}{\tau_I} \end{pmatrix}. \quad (14)$$

因此

$$\text{tr } J = \frac{M_{EE} - 1}{\tau_E} - \frac{1 + M_{II}}{\tau_I}, \quad (15)$$

$$\det J = \frac{M_{EI}M_{IE} - (M_{EE} - 1)(1 + M_{II})}{\tau_E \tau_I}. \quad (16)$$

以 τ_I 为分叉参数

Hopf 分叉要求

$$\text{tr } J = 0, \quad \det J > 0.$$

由 $\text{tr } J = 0$ 得到临界值

$$\tau_I^H = \frac{\tau_E(1 + M_{II})}{M_{EE} - 1} \quad (17)$$

其中需要 $M_{EE} > 1$. 同时还需要

$$M_{EI}M_{IE} > (M_{EE} - 1)(1 + M_{II}) \quad (18)$$

保证 $\det J > 0$. 横截性条件为

$$\frac{d}{d\tau_I} \operatorname{tr} J = \frac{1 + M_{II}}{\tau_I^2} \neq 0, \quad (19)$$

所以在上述条件下出现 Hopf 分叉.

取一组数值

$$\tau_E = 1, \quad M_{EE} = 2, \quad M_{EI} = 3, \quad M_{IE} = 3, \quad M_{II} = 1, \quad h_E = 2, \quad h_I = -1.$$

则

$$\tau_I^H = 2, \quad \omega_H = \sqrt{\det J|_{\tau_I=2}} = \sqrt{\frac{7}{2}} \approx 1.8708. \quad (20)$$

此时平衡点为

$$(v_E^*, v_I^*) = (1, 1),$$

并且两个阈值内部输入均为正, 所以该平衡点确实位于当前线性区域内.

数值模拟结果如下表. 可以看出, 当 $\tau_I < 2$ 时, 轨道收敛到平衡点; 当 $\tau_I > 2$ 时, 平衡点失稳, 并出现明显振荡.

τ_I	$\operatorname{tr} J$	$\det J$	后半段 v_E 范围	后半段 v_I 范围
1.0	-1.0000	7.0000	[1.0000, 1.0000]	[1.0000, 1.0000]
1.8	-0.1111	3.8889	[0.9983, 1.0016]	[0.9988, 1.0013]
2.0	0	3.5000	[0.3877, 1.5811]	[0.5891, 1.4109]
2.2	0.0909	3.1818	[0.2703, 1.7194]	[0.5398, 1.4985]
3.0	0.3333	2.3333	[0.0807, 2.1094]	[0.4898, 1.7204]

以 h_E 为分叉参数

在同一个激活区域内, Jacobian 矩阵 J 与 h_E 无关. 因此 h_E 只改变平衡点位置, 不改变局部线性化特征值. 也就是说, 在该阈值线性模型的固定激活区域内, h_E 不能引起经典 Hopf 分叉.

仍取上面的参数并固定 $h_I = -1$, 平衡点为

$$v_E^* = \frac{2h_E + 3}{7}, \quad v_I^* = \frac{3h_E + 1}{7}. \quad (21)$$

当 $h_E > -\frac{1}{3}$ 时, 平衡点位于正区域内. 只要不跨过阈值边界, 特征值不随 h_E 改变. 因此, h_E 的改变可能导致的是阈值边界上的非光滑变化, 而不是经典意义下的 Hopf 分叉.

3 第 4 题: 多神经元方向刺激的近似 MVUB 估计

假设在观测时间窗 $[0, T_0]$ 内, 第 a 个神经元对方向 s 的条件发放率为

$$f_a(s) = \exp\left(-\frac{(s - s_a)^2}{2\sigma_a^2}\right), \quad a = 1, \dots, N. \quad (22)$$

在题设假设下, 各神经元相互独立且发放为 Poisson 过程. 因此观测到的 spike time 在给定计数后不再额外携带关于 s 的信息, 核心统计量可以取为 spike count

$$n_a = N_a(T_0), \quad n_a | s \sim \operatorname{Pois}(T_0 f_a(s)).$$

于是似然函数为

$$L(s) = \prod_{a=1}^N \frac{(T_0 f_a(s))^{n_a}}{n_a!} \exp[-T_0 f_a(s)]. \quad (23)$$

对数似然为

$$\ell(s) = \sum_{a=1}^N n_a \log f_a(s) - T_0 \sum_{a=1}^N f_a(s) + C, \quad 0 \leq s \leq 180^\circ. \quad (24)$$

因此可以取近似 MVUB 估计为最大似然估计

$$\hat{s} = \arg \max_{0 \leq u \leq 180^\circ} \left\{ \sum_{a=1}^N n_a \log f_a(u) - T_0 \sum_{a=1}^N f_a(u) \right\}. \quad (25)$$

这里说“近似”是因为有限样本下一般很难写出严格的 MVUB 闭式解；但在 Poisson 独立模型下,MLE 是一致且渐近有效的,样本量较大时可达到 Cramer–Rao 下界,因此可作为近似 MVUB 解.

由于

$$f'_a(s) = f_a(s) \left(-\frac{s - s_a}{\sigma_a^2} \right), \quad (26)$$

Poisson 模型的 Fisher 信息为

$$I(s) = T_0 \sum_{a=1}^N \frac{(f'_a(s))^2}{f_a(s)} = T_0 \sum_{a=1}^N f_a(s) \frac{(s - s_a)^2}{\sigma_a^4}. \quad (27)$$

因此平方误差的理论近似为

$$\mathbb{E}[(\hat{s} - s)^2] \approx \frac{1}{I(s)} = \left[T_0 \sum_{a=1}^N f_a(s) \frac{(s - s_a)^2}{\sigma_a^4} \right]^{-1}. \quad (28)$$

数值验证中取

$$N = 36, \quad s_a = 0, 5, 10, \dots, 175, \quad \sigma_a = 20^\circ, \quad T_0 = 2, \quad s = 75^\circ.$$

计算得到

$$I(75^\circ) = 0.05009, \quad \frac{1}{I(75^\circ)} = 19.9646. \quad (29)$$

Monte Carlo 模拟 2000 次得到

$$\bar{\hat{s}} = 75.0514, \quad \text{Bias} = 0.0514, \quad \text{Var}(\hat{s}) = 20.5783, \quad \text{MSE} = 20.5707. \quad (30)$$

模拟 MSE 与 Cramer–Rao 下界非常接近,说明该估计在本例中是较好的近似最小方差无偏估计.

4 第 5 题: 音频盲源分离与质量评估

附件中给出了三个混合音频文件,记为

$$x_1(t), \quad x_2(t), \quad x_3(t).$$

把它们组成矩阵

$$X(t) = \begin{pmatrix} x_1(t) & x_2(t) & x_3(t) \end{pmatrix}.$$

盲源分离的基本模型为

$$X = AS, \quad (31)$$

其中 S 是未知源信号, A 是未知混合矩阵. 由于附件中没有给出干净源信号, 所以不能计算有监督的 SDR、SIR 等指标. 这里采用无监督评价指标: 重构误差、分离后信号之间的相关性、平方信号相关性以及峰度.

本文比较两种算法:

1. **PCA 白化法**: 只利用二阶统计量, 使输出信号两两不相关.
2. **FastICA**: 在白化基础上进一步最大化非高斯性, 从而尽量恢复统计独立的源信号.

设分离信号为 $\hat{S} = (\hat{s}_1, \hat{s}_2, \hat{s}_3)$. 评价指标取

$$E_{\text{rec}} = \frac{\|X - A\hat{S}\|_F}{\|X\|_F}, \quad (32)$$

$$C_1 = \text{mean}_{i < j} |\text{corr}(\hat{s}_i, \hat{s}_j)|, \quad (33)$$

$$C_2 = \text{mean}_{i < j} |\text{corr}(\hat{s}_i^2, \hat{s}_j^2)|. \quad (34)$$

其中 C_1 衡量普通线性相关性, C_2 用来粗略衡量非线性依赖. 若分离效果更好, 则 C_2 应更小.

程序运行结果如下.

算法	E_{rec}	C_1	C_2	平均绝对峰度
PCA	1.80×10^{-16}	1.19×10^{-16}	0.1273	0.3820
FastICA	2.40×10^{-16}	3.19×10^{-16}	0.0246	1.1303

两种方法的重构误差都接近机器精度, 这是因为都使用了三个分量进行重构. PCA 已经能做到线性去相关, 所以 C_1 很小. 但是 PCA 分离后的平方信号相关性 $C_2 = 0.1273$, 明显大于 FastICA 的 0.0246. 同时 FastICA 的平均绝对峰度更大, 说明其分离结果更加非高斯, 更符合独立源信号的特征. 因此, 本题中 FastICA 的盲源分离效果优于 PCA.

5 第 7 题: 用强化学习思想求解迷宫最短路径

把迷宫看成一个确定性 Markov decision process. 状态 s 是可通行像素点, 动作集合为

$$\mathcal{A} = \{\text{up}, \text{down}, \text{left}, \text{right}\}.$$

若动作撞墙, 则该动作不可取; 若动作可以走到相邻像素, 则奖励设为

$$r(s, a) = -1.$$

终点状态 g 的价值为 $V(g) = 0$. 在折扣因子 $\gamma = 1$ 的 episodic 情形下, Bellman 最优方程为

$$V(s) = \max_{a \in \mathcal{A}(s)} \{-1 + V(s')\}, \quad (35)$$

其中 s' 是从 s 采取动作 a 后到达的状态. 由于每走一步的奖励都是 -1 , 最大化累计奖励等价于最小化路径长度. 因此, 收敛后的贪心策略给出的就是最短路径.

在实现中, 先对 maze.jpg 做预处理: 白色区域视作墙, 黑色区域视作可通行区域, 并对墙体做轻微膨胀来消除图像中斜线纹理造成的小孔. 黄色点自动识别为出发点. 程序检测到出发点像素坐标为

$$S = (558, 402).$$

这里坐标采用图像坐标, 即 (x, y) . 任选一个终点

$$G = (320, 249).$$

程序给出的最短路径长度为

$$\boxed{6115 \text{ pixel steps}}. \quad (36)$$

红色曲线为最短路径, 黄色点为出发点, 绿色点为终点.

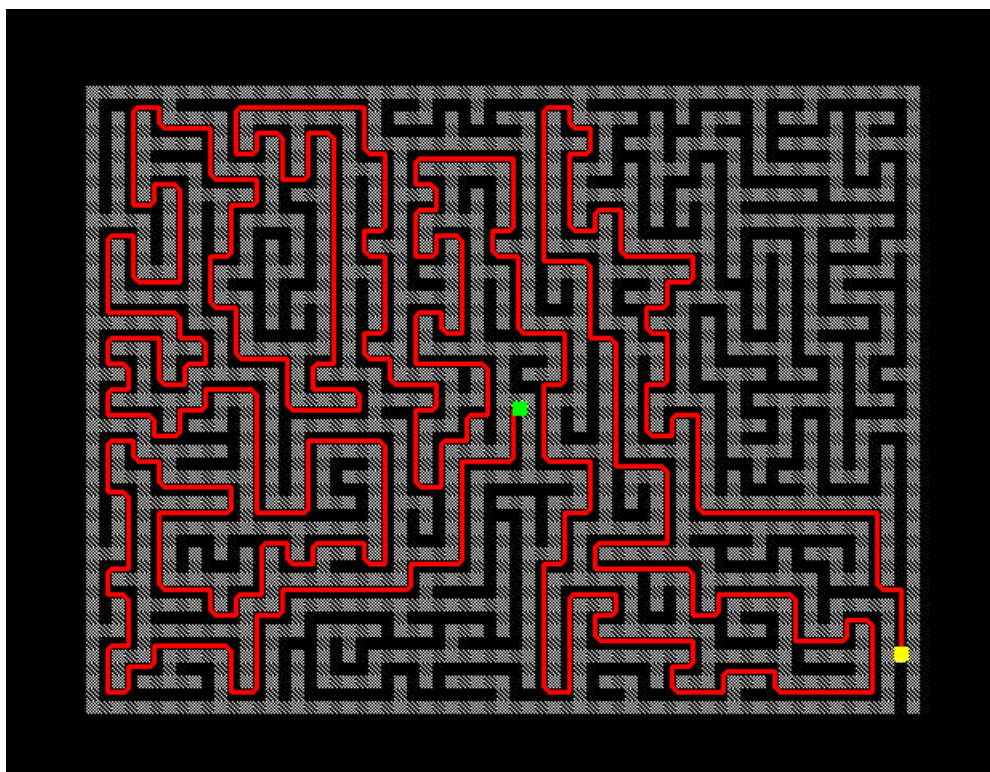


图 1: 迷宫最短路径结果示意图

A Python 代码

```

1 import glob
2 from pathlib import Path
3 from collections import deque
4
5 import numpy as np
6 from scipy.integrate import solve_ivp
7 from scipy.io import wavfile
8 from scipy.ndimage import binary_dilation
9 from scipy.stats import kurtosis
10 from PIL import Image
11
12
13 def problem3_hopf_simulation():
14     tauE = 1.0
15     MEE, MEI, MIE, MII = 2.0, 3.0, 3.0, 1.0
16     hE, hI = 2.0, -1.0
17
18     tauI_H = tauE * (1 + MII) / (MEE - 1)
19     omega_H = np.sqrt((MEI * MIE - (MEE - 1) * (1 + MII)) / (tauE * tauI_H))
20     print("tauI_H =", tauI_H, "omega_H =", omega_H)
21
22     def rhs(t, y, tauI):
23         vE, vI = y
24         zE = MEE * vE - MEI * vI + hE
25         zI = MIE * vE - MII * vI + hI
26         return [(-vE + max(zE, 0.0)) / tauE,
27                 (-vI + max(zI, 0.0)) / tauI]
28
29     for tauI in [1.0, 1.8, 2.0, 2.2, 3.0]:
30         sol = solve_ivp(lambda t, y: rhs(t, y, tauI), [0, 200], [1.2, 0.8],
31                         max_step=0.05, rtol=1e-7, atol=1e-9)
32         tail = sol.y[:, sol.t > 100]
33         print("tauI=", tauI,
34               "min=", tail.min(axis=1),
35               "max=", tail.max(axis=1),
36               "std=", tail.std(axis=1))
37
38
39 def problem4_direction_mle(seed=0, runs=2000):
40     rng = np.random.default_rng(seed)
41     N = 36
42     pref = np.linspace(0.0, 180.0, N, endpoint=False)
43     sigma = 20.0 * np.ones(N)
44     Tobs = 2.0
45     s_true = 75.0
46     grid = np.linspace(0.0, 180.0, 1801)
47

```

```

48     def rates(s):
49         return np.exp(-0.5 * ((s - pref) / sigma) ** 2)
50
51     F_grid = np.exp(-0.5 * ((grid[:, None] - pref[None, :]) / sigma[None, :]) ** 2)
52     logF_grid = np.log(F_grid + 1e-300)
53
54     def mle(counts):
55         ll = (counts[None, :] * logF_grid).sum(axis=1) - Tobs * F_grid.sum(axis=1)
56         return grid[np.argmax(ll)]
57
58     fisher = Tobs * np.sum(rates(s_true) * (s_true - pref) ** 2 / sigma ** 4)
59     estimates = []
60     for _ in range(runs):
61         counts = rng.poisson(Tobs * rates(s_true))
62         estimates.append(mle(counts))
63     estimates = np.array(estimates)
64     print("Fisher information =", fisher)
65     print("CRLB =", 1.0 / fisher)
66     print("mean =", estimates.mean())
67     print("bias =", estimates.mean() - s_true)
68     print("variance =", estimates.var(ddof=1))
69     print("MSE =", np.mean((estimates - s_true) ** 2))
70
71
72 def problem5_bss_audio():
73     from sklearn.decomposition import FastICA, PCA
74
75     wavs = sorted(Path('.').rglob('110000001mix*.wav'))
76     if len(wavs) < 3:
77         wavs = sorted(Path('/mnt/data').rglob('110000001mix*.wav'))
78     wavs = wavs[:3]
79     if len(wavs) != 3:
80         raise FileNotFoundError('Need three mixed wav files named 110000001mix*.wav')
81
82     signals = []
83     fs0 = None
84     for p in wavs:
85         fs, x = wavfile.read(str(p))
86         if fs0 is None:
87             fs0 = fs
88         if x.dtype == np.uint8:
89             x = (x.astype(np.float64) - 128.0) / 128.0
90         else:
91             x = x.astype(np.float64)
92             x = x / (np.max(np.abs(x)) + 1e-12)
93         signals.append(x)
94     X = np.vstack(signals).T
95     X = X - X.mean(axis=0)

```



```

96
97     pca = PCA(n_components=3, whiten=True, random_state=0)
98     S_pca = pca.fit_transform(X)
99     ica = FastICA(n_components=3, whiten='unit-variance', random_state=0,
100                  max_iter=2000, tol=1e-5)
101     S_ica = ica.fit_transform(X)
102
103     def standardize(S):
104         return (S - S.mean(axis=0)) / (S.std(axis=0) + 1e-12)
105
106     def metrics(name, S, Xrec):
107         Z = standardize(S)
108         C = np.corrcoef(Z.T)
109         C2 = np.corrcoef((Z ** 2).T)
110         off = np.mean(np.abs(C[np.triu_indices(3, 1)]))
111         off2 = np.mean(np.abs(C2[np.triu_indices(3, 1)]))
112         kur = np.mean(np.abs(kurtosis(Z, fisher=True, axis=0)))
113         rec = np.linalg.norm(X - Xrec) / np.linalg.norm(X)
114         print(name, 'rec=', rec, 'mean_abs_corr=', off,
115               'mean_abs_sqcorr=', off2, 'mean_abs_kurtosis=', kur)
116         return rec, off, off2, kur
117
118     metrics('PCA', S_pca, pca.inverse_transform(S_pca))
119     metrics('FastICA', S_ica, ica.inverse_transform(S_ica))
120
121     outdir = Path('bss_outputs')
122     outdir.mkdir(exist_ok=True)
123     for name, S in [('pca', S_pca), ('fastica', S_ica)]:
124         for j in range(3):
125             y = S[:, j]
126             y = y / (np.max(np.abs(y)) + 1e-12)
127             wavfile.write(str(outdir / f'{name}_source{j+1}.wav'), fs0,
128                           (32767 * y).astype(np.int16))
129
130
131     def problem7_maze_solver(target_xy=(320, 249)):
132         candidates = list(Path('.').rglob('maze.jpg')) + list(Path('/mnt/data').rglob('
133             maze.jpg'))
134         if not candidates:
135             raise FileNotFoundError('maze.jpg not found')
136         maze_path = candidates[0]
137         img = Image.open(str(maze_path)).convert('RGB')
138         arr = np.array(img)
139
140         bright = (arr[:, :, 0] > 100) & (arr[:, :, 1] > 100) & (arr[:, :, 2] > 100)
141         ys, xs = np.where(bright)
142         x0, x1, y0, y1 = xs.min(), xs.max(), ys.min(), ys.max()
143         inside = np.zeros(bright.shape, dtype=bool)
144         inside[y0:y1 + 1, x0:x1 + 1] = True

```

```

144
145     score = arr[:, :, 0].astype(int) + arr[:, :, 1].astype(int) - 2 * arr[:, :, 2].
        astype(int)
146     yellow = (score > 120) & (arr[:, :, 0] > 120) & (arr[:, :, 1] > 100)
147     ys0, xs0 = np.where(yellow)
148     start_yx = (int(round(ys0.mean()))), int(round(xs0.mean()))
149
150     wall = bright.copy()
151     wall[yellow] = False
152     wall = binary_dilation(wall, iterations=2)
153     free = inside & (~wall)
154     sy, sx = start_yx
155     free[sy - 3:sy + 4, sx - 3:sx + 4] = True
156
157     tx, ty = target_xy
158     target_yx = (ty, tx)
159     if not free[target_yx]:
160         print('The selected target is on a wall after preprocessing.')
161         return None
162
163     H, W = free.shape
164     dist = -np.ones((H, W), dtype=np.int32)
165     prev = np.full((H, W, 2), -1, dtype=np.int16)
166     q = deque([start_yx])
167     dist[start_yx] = 0
168     while q:
169         y, x = q.popleft()
170         if (y, x) == target_yx:
171             break
172         for dy, dx in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
173             ny, nx = y + dy, x + dx
174             if 0 <= ny < H and 0 <= nx < W and free[ny, nx] and dist[ny, nx] < 0:
175                 dist[ny, nx] = dist[y, x] + 1
176                 prev[ny, nx] = [y, x]
177                 q.append((ny, nx))
178
179     if dist[target_yx] < 0:
180         print('Target is unreachable.')
181         return None
182
183     path = []
184     y, x = target_yx
185     while y >= 0 and x >= 0:
186         path.append((y, x))
187         py, px = prev[y, x]
188         if py < 0:
189             break
190         y, x = int(py), int(px)
191     path = path[::-1]

```

```
192     print('start (x,y)=', (sx, sy), 'target (x,y)=', target_xy,
193           'shortest length=', dist[target_yx])
194
195     overlay = arr.copy()
196     pm = np.zeros((H, W), dtype=bool)
197     for y, x in path:
198         pm[y, x] = True
199     pm = binary_dilation(pm, iterations=1)
200     overlay[pm] = [255, 0, 0]
201     overlay[sy - 4:sy + 5, sx - 4:sx + 5] = [255, 255, 0]
202     overlay[ty - 4:ty + 5, tx - 4:tx + 5] = [0, 255, 0]
203     Image.fromarray(overlay).save('maze_solution.png')
204     return path
205
206
207 if __name__ == '__main__':
208     problem3_hopf_simulation()
209     problem4_direction_mle()
210     problem5_bss_audio()
211     problem7_maze_solver(target_xy=(320, 249))
```